1.3

```cpp
#include <iostream>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>

using namespace std;
/*
 * Проблема читачів та письменників.
 * У нас є читачі, декілька читачей можуть читати одночасно, а серед письменників
 * писати може тільки один. В цій задачі потрібно скоординувати доступ до
 * спільного ресурсу для багатьох потоків, деякі з яких можуть читати ресурс,
 * а інші - писати в нього. Основне правило полягає в тому, що письменник не може писати,
 * коли будь-який читач читає, і навпаки.
*/

#define READERS_NUM 4      //кількістьть читачів
#define WRITERS_NUM 2      //кількість письменників
#define READERS_VISIT 1     //кількість відвідувань кожного читача
#define  WRITERS_VISIT 1     //кількість відвідувань кожного письменника


/*
 * Далі ми оголошуємо змінні для семафорів (ReadWriteSem і VariablesSem),
 * лічильників читачів (r_count) і письменників (w_count), а також флагу (flag).
 */
sem_t ReadWriteSem;    //надає доступ або читачам, або одному письменнику
sem_t VariablesSem;    //надає доступ до лічильника та виводу в консоль
int r_count = 0;   //кількість читачів
int w_count = 0;   //кількість письменників
bool flag = false;   //флаг
```

main

OS 3

```cpp
void* Readers(void* data) //функція читачів
{
    int num = *(int*)data;    //номер читача
    sleep( seconds: rand() % 5); // створення випадкового затримки перед початком роботи кожного читача.
    int counter = 0; // лічильник

    //виконується, доки лічильник counter не досягне максимальної кількості відвідувань
    while (counter < READERS_VISIT)
    {
        /*
         * перевіряють флаг flag, який вказує, чи є письменник у черзі.
         * Якщо флаг встановлено (тобто є письменник), читач очікує
         */
        if (flag)
        {
            //якщо у черзі письменник, читач не ставає в неї
            sleep( seconds: rand() % 10);
            continue;
        }
        /*
         * Спочатку читач блокує семафор VariablesSem.
         * Потім перевіряється, чи немає читачів в базі (r_count == 0).
         * Якщо це так, семафор VariablesSem розблоковується,
         * щоб інші читачі могли увійти, та чекається на семафор ReadWriteSem,
         * який контролює доступ письменників до бази.
         * Після отримання доступу до ReadWriteSem, семафор VariablesSem
         * знову блокується для оновлення значень змінних.
         */
        sem_wait( sem: &VariablesSem); //блокування семафору
        if (r_count == 0)
        {        //якщо більше читачів немає
```

```
    sem_post( sem: &VariablesSem);    //розблоковування семафору
    sem_wait( sem: &ReadWriteSem);  //читач заходить, або чекає коли письменник закінчить
    sem_wait( sem: &VariablesSem);
}
/*
 * повідомлення про початок відвідування читача та розблоковують семафор
 * VariablesSem, щоб інші читачі могли увійти.
 */
r_count++;  //читач зайшов
cout << "Reader " << num << " start (" << counter + 1 << " time)." << " Total: " << r_count << endl;
sem_post( sem: &VariablesSem);      //розблоковуємо семафор доступу до спільних змінних

// симулює час, який читач витрачає на "читання"
sleep( seconds: rand() % 10);        //очікування поки читач "читає"



/*
 * знову блокують семафор VariablesSem і зменшують лічильник читачів r_count.
 * Потім виводиться повідомлення про закінчення відвідування читача та
 * розблоковується семафор VariablesSem.
 */
sem_wait( sem: &VariablesSem); //заблоковуємо семафор доступу до спільних змінних
r_count--;            //читач виходить з бази
cout << "Reader " << num << " stop (" << counter + 1 << " time)." << " Total: " << r_count << endl;
sem_post( sem: &VariablesSem);



/*
 *перевіряють, чи немає жодного читача в базі (r_count == 0).
 * Якщо це так, семафор ReadWriteSem розблоковується,
 * щоб дозволити письменнику увійти
```

_3

```
95          if (r_count == 0)  //якщо читачів в базі немає
96          {
97              sem_post( sem: &ReadWriteSem);  //дозволяємо заходит письменнику
98          }
99          counter++;
100
101         sleep( seconds: rand() % 2);
102         }
103     return 0;
104 }
105
106 void* Writers(void* data) //функція письменників
107 {
108     int num = *(int*)data; //номер письменника
109     int counter = 0;
110
111     while (counter < WRITERS_VISIT)
112     {
113         sleep( seconds: rand() % 10);
114
115         /*
116          *що повідомляє читачам, що письменник очікує доступу до бази даних, тому вони повинні звільнити базу
117          * та не ставати у чергу. Після цього письменник чекає на семафор ReadWriteSem, який контролює доступ
118          * до бази даних. Після отримання доступу флаг flag встановлюється у значення false, що дозволяє читачам
119          * знову ставати у чергу
120          */
121         flag = true;  //повідомлення для читачів, що вони мають звільнити базу та не ставати у чергу
122         sem_wait( sem: &ReadWriteSem);  //письменник чекає
123         flag = false;    //читачі можуть ставати у чергу
124
125         /*
```

main

```
            * виводять повідомлення про початок роботи письменника, сплять протягом 2 секунд
            * (симулюючи час написання в базу даних), зменшують лічильник письменників w_count
            * і виводять повідомлення про завершення роботи письменника. Нарешті, викликається
            * sem_post для відкриття доступу до бази даних для читачів. Лічильник counter збільшується,
            * і цикл повторюється для наступного відвідування письменника.
            */
           w_count++;
           cout << "Writer " << num << " start (" << counter + 1 << " time)." << " Total: " << w_count << endl;
           sleep( seconds: 2);
           w_count--;        //письменник закінчив
           cout << "Writer " << num << " stop (" << counter + 1 << " time)." << " Total: " << w_count << endl;
           sem_post( sem: &ReadWriteSem);    //відкриття доступу читачам

           counter++;
           }
       return 0;
}

int main()
{
    srand( seed: time( timer: NULL));

    //ініціалізація семафорів VariablesSem та ReadWriteSem
    sem_init( sem: &VariablesSem,  pshared: 0,  value: 1);
    sem_init( sem: &ReadWriteSem,  pshared: 0,  value: 1);

    //створення потоків читачів та письменників
    /*
     * масиви ReaderThreads та WriterThreads, які
     * будуть містити ідентифікатори потоків читачів
     * та письменників відповідно.
     */
    pthread_t ReaderThreads[READERS_NUM];
    pthread_t WriterThreads[WRITERS_NUM];

    /*
     * цикл створює потоки для читачів. Для кожного читача створюється новий
     * об'єкт типу int, який містить номер читача, і передається у функцію
     * Readers за допомогою параметру temp. Функція pthread_create створює новий потік,
     * який виконує функцію Readers, із вказаним параметром temp.
     */
    for (int i = 0; i < READERS_NUM; i++)
    {
        int* temp = new int(i);

        pthread_create( newthread: &ReaderThreads[i],  attr: NULL,  start_routine: Readers,  arg: temp);
    }
    /*
     * Аналогічно до попереднього циклу, цей цикл створює потоки для письменників.
     */
    for (int i = 0; i < WRITERS_NUM; i++)
    {
        int* temp = new int(i);

        pthread_create( newthread: &WriterThreads[i],  attr: NULL,  start_routine: Writers,  arg: temp);
    }

    //очікування завершення потоків
    /*
     * цикли виконують очікування завершення усіх потоків читачів і письменників.
     * Функція pthread_join блокує виконання поточного потоку до тих пір, поки не завершиться потік, переданий як аргумент
```

```cpp
    /*
     * Аналогічно до попереднього циклу, цей цикл створює потоки для письменників.
     */
    for (int i = 0; i < WRITERS_NUM; i++)
    {
        int* temp = new int(i);

        pthread_create( newthread: &WriterThreads[i],  attr: NULL,  start_routine: Writers,  arg: temp);
    }

    //очікування завершення потоків
    /*
     * цикли виконують очікування завершення усіх потоків читачів і письменників.
     * Функція pthread_join блокує виконання поточного потоку до тих пір, поки не завершиться потік, переданий як аргумент
     */
    for (int i = 0; i < READERS_NUM; i++)
    {
        pthread_join( th: ReaderThreads[i],  thread_return: NULL);
    }
    for (int i = 0; i < WRITERS_NUM; i++)
    {
        pthread_join( th: WriterThreads[i],  thread_return: NULL);
    }

    //видаляємо семафори
    sem_destroy( sem: &ReadWriteSem);
    sem_destroy( sem: &VariablesSem);

    return 0;
}
```